

เจาะลึกอัลกอริทึม K-Partition Subset Sum (Problem 84)

มาตรฐานการอธิบายโค้ดฉบับ Modern Technical Minimalist TH

1. ปัญหาคืออะไร? (Problem Definition)

โจทย์ต้องการให้เราตรวจสอบว่าลิสต์ของตัวเลข `nums` สามารถแบ่งออกเป็น `k` กลุ่ม โดยที่ ทุกกลุ่มมีผลรวมเท่ากัน ได้หรือไม่? และในกรณีที่ทำได้ ให้แสดงรายการของสมาชิกในแต่ละกลุ่มออกมาด้วย

ข้อกำหนดสำคัญ (Constraints):

- จำนวนสมาชิกใน `nums` อยู่ระหว่าง 1 ถึง 16
- ตัวเลขทุกตัวเป็นจำนวนเต็มบวก
- ผลรวมทั้งหมดหาร `k` ลงตัวเสมอ

2. อธิบายโค้ด ทีละส่วน (Sectional Reasoning)

ส่วนที่ 1: การเตรียมตัวและตรวจสอบเบื้องต้น

เราคำนวณ `target` ของแต่ละกลุ่ม และเรียงลำดับจาก มากไปน้อย เพื่อให้เจอทางเลือกที่ผิดพลาดได้เร็วขึ้น (Fail Fast)

ส่วนที่ 2: กลไก Backtracking

เราใช้ฟังก์ชัน `backtrack(start_index, k_idx, current_sum)` เพื่อจำลองการหยิบตัวเลขใส่กล่องทีละใบ:

- สำเร็จ: เมื่อกล่องปัจจุบันเต็ม ให้ไปเริ่มกล่องใหม่
- จบงาน: เมื่อทุกกล่อง (`k`) เต็มครบถ้วน
- ถอยกลับ (Backtrack): หากหยิบตัวเลขแล้วไปต่อไม่ได้ ให้เอาตัวเลขนั้นออกแล้วลองตัวถัดไป

3. การวิเคราะห์ประสิทธิภาพ (Complexity)

- Time Complexity:** $O(k \cdot 2^N)$ แม้ดูเหมือนจะช้า แต่การตัดกิ่ง (Pruning) ทำให้ทำงานได้จริงในขนาด $N=16$
- Space Complexity:** $O(N)$ สำหรับจัดเก็บสถานะและ Call Stack

ตารางสรุปบทบาทหน้าที่

ส่วนประกอบ	หน้าที่	ผลลัพธ์ที่ได้
<code>sort(reverse=True)</code>	จัดลำดับความสำคัญ	ค้นหาได้เร็วขึ้นแบบก้าวกระโดด
<code>used array</code>	หน่วยความจำหลัก	ป้องกันการใช้เลขซ้ำซ้อน
<code>subsets.append()</code>	บันทึกหลักฐาน	ได้รายการสมาชิกในแต่ละกลุ่ม
<code>current_sum == 0: break</code>	การตัดกิ่งขั้นสูง	ข้ามเส้นทางที่ไม่มีวันสำเร็จ